National University of Computer and Emerging Sciences

Phase 1 Project Proposal: NeuroScan

Group ID: F26-07

	Name	**Roll.no**
1.	Moeed (Team Lead)	23L0686
2.	Moiz Ahmed	23L-2528
3.	M.Auon Naseer	23L-0643

1. Dataset Source & Description

The primary dataset utilized for the NeuroScan project is the **Brain Tumor MRI Dataset**, acquired from the Kaggle data repository

https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

This dataset consists of T1-weighted, contrast-enhanced Magnetic Resonance Imaging (MRI) scans. During the clinical acquisition of these scans, a Gadolinium contrast dye was utilized to chemically highlight abnormal vascular structures, causing tumor masses to appear hyperintense (bright) against healthy brain tissue.

The dataset is categorized into four distinct multi-class labels representing specific types of intracranial anomalies, alongside a control group:

1. **Glioma Tumors**
2. **Meningioma Tumors**
3. **Pituitary Tumors**
4. **No Tumor (Healthy Control)**

2. Data Collection Method

The data was collected via direct download from the Kaggle repository as a compressed archive. Upon extraction into our local development environment, the dataset was found to be pre-structured into dedicated `Training` and `Testing` directories, with subclass folders for each of the four categories.

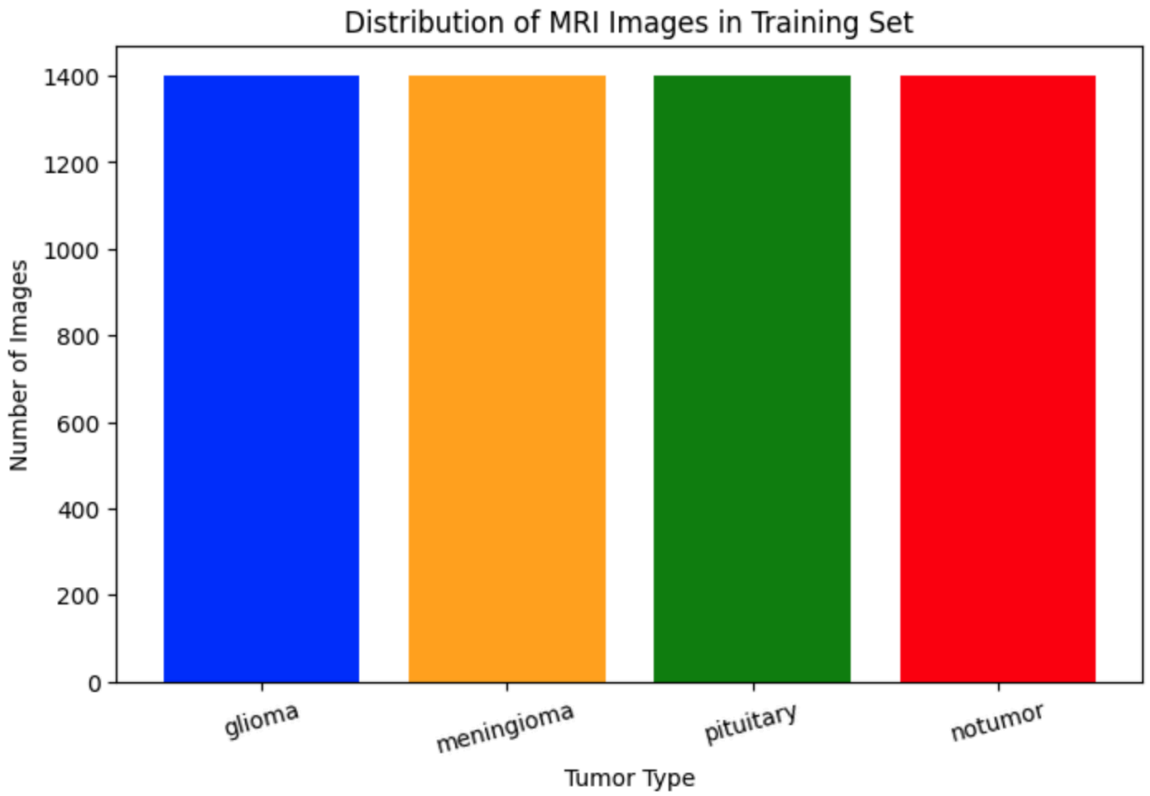
For the data loading phase, we utilized Python's built-in `os` library to programmatically traverse the directory trees. Instead of relying on manual file handling, we implemented a custom data generator function that dynamically reads the image paths, pairs them with their respective class indices (0 for glioma, 1 for meningioma, etc.), and loads the files directly into system memory using the Python Imaging Library (Pillow).

3. Brief EDA (Key Insights)

Exploratory Data Analysis (EDA) was conducted to understand the underlying distribution and visual characteristics of our training data before feeding it into a neural network.

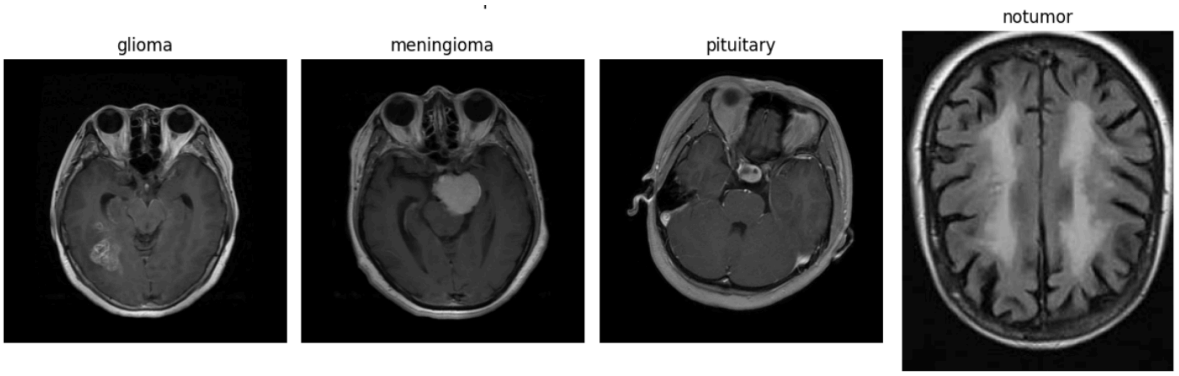
Key Insight: Perfect Class Balance The most critical insight derived from our EDA is the presence of a perfectly balanced dataset. Using a custom iteration script, we mapped the distribution of the training directory and found exactly 1,400 images for each of the four classes, yielding a total of 5,600 training images.

This perfect balance is highly advantageous. In medical imaging AI, class imbalance (e.g., having 5,000 Gliomas but only 200 Pituitary scans) often causes a Convolutional Neural Network (CNN) to develop a severe diagnostic bias. Because our data is perfectly distributed, we can proceed with training without needing to implement complex oversampling techniques like SMOTE.



Sample MRI Scans from Each Class

To visually verify the data integrity before processing, we extracted the first image from each sub-directory and plotted them side-by-side. This confirmed that the images successfully retained their contrast enhancement and structural details:



4. Preprocessing Steps

Because Convolutional Neural Networks (CNNs) perform strict matrix multiplications, they require highly standardized, uniform inputs. Raw image files vary in dimensions and color profiles, meaning they cannot be fed directly into an AI. We applied a strict three-step preprocessing pipeline using Pillow (PIL) and NumPy to prepare the data:

1. **RGB Conversion (`.convert('RGB')`):** Medical images often have inconsistent color channels (some grayscale, some varying color depths). We forced all images

into a standard 3-channel RGB format to ensure the input tensor depth remains uniform.

2. **Standardized Resizing (`.resize((224, 224))`):** Original MRI scans varied in their native resolutions. We mathematically resized every single image to a strict `224x224` pixel grid. This specific dimension was chosen as it is the optimal, required input size for standard, pre-trained CNN architectures like ResNet and EfficientNet.
3. **Pixel Normalization (`/ 255.0`):** Raw digital images consist of integer pixel values ranging from 0 (pure black) to 255 (pure white). Feeding large numbers into a neural network causes mathematical instability during backpropagation. We converted the resized images into NumPy arrays and divided all values by 255.0, scaling the entire dataset to a normalized float range between `[0.0, 1.0]`. This step drastically improves gradient descent stability and model convergence speed.

5. Final Dataset Summary

To verify our preprocessing pipeline without exhausting local system RAM, we processed a uniform subset of 500 images from each class (2,000 images total). Following the pipeline, our data was successfully transformed from raw `.jpg` files into a multi-dimensional mathematical tensor, fully prepared for Phase 2 model implementation.

- **Pipeline Verification Batch Size:** 2,000 images (500 per class)
- **Final Training Array Shape:** `(2000, 224, 224, 3)`
    - **2000:** Total number of successfully processed images in the array.
    - **224, 224:** The standardized, uniform height and width of every image.
    - **3:** The standardized RGB color channels.

```
Preprocessing Complete
Final Training Data Shape: (2000, 224, 224, 3)
```

6.Conclusion:

Github link for his repo: **https://github.com/Moeeeeed/neuroscan**

Paper Links: * [SpringerOpen Official Publication]

- [PMC PubMed Central Full Text]